ULTIMATE LLAMA.CPP GUIDE & MANUAL

Optimized for Single-Node & Distributed Computing Clusters (ThinkStack Architecture)

Target System: Ubuntu 24.04 / ThinkPad T470 (i5-7300U, 24GB RAM, Intel HD 620 Vulkan)
Updated Features: `ggml-rpc-server`, `--load-mode mlock`, `--reasoning`, Single Worker Offloading

PHASE 1: INSTALLATION & COMPILATION (UNIVERSAL VULKAN + STATIC + RPC)

This phase builds standalone binaries (`llama-cli` , `llama-server` , `ggml-rpc-server`) with Vulkan GPU acceleration and Distributed RPC backend support.

1. **Install required dependencies:**

```
sudo add-apt-repository universe -y && sudo apt update
sudo apt install build-essential git cmake jq libvulkan-dev vulkantools glslang-tools gl
```

2. **Clone and compile llama.cpp with Vulkan + Static + RPC flags:**

```
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
rm -rf build # Clean start
cmake -B build -DGGML_VULKAN=ON -DGGML_RPC=ON -DBUILD_SHARED_LIBS=OFF -DGGML_STATIC=ON
cmake --build build --config Release -j $(nproc)
sudo cp build/bin/llama-* build/bin/ggml-rpc-server /usr/local/bin/
```

PHASE 2: MAPPING OLLAMA MODELS TO LLAMA.CPP

Ollama stores GGUF models as raw blobs without file extensions. You can map and run them directly in llama.cpp:

1. **View model manifest:**

```
cat /media/osimage/INT-WORKERT470/OLLAMA_MODELS_STORAGE/manifests/registry.ollama.ai/lib
```

2. **Locate the digest hash in the `blobs/` directory:**
   Ollama filenames replace the colon `sha256:59bb50 ...` with a hyphen: `sha256-59bb50 ...`

```
/media/osimage/INT-WORKERT470/OLLAMA_MODELS_STORAGE/blobs/sha256-59bb50 ...
```

PHASE 3: RUNNING THE LOCAL SERVER FOR VS CODE AUTOCOMPLETE

Provides zero-latency "Ghost Text" autocomplete suggestions and chat via the **Continue** extension or API clients.

```
llama-server \
  -m /media/osimage/INT-WORKERT470/OLLAMA_MODELS_STORAGE/blobs/sha256-YOUR-GEMMA3-HASH \
  --port 9090 \
  -t 2 \
  -ngl 99 \
  -c 4096 \
  --load-mode mlock \
  --reasoning auto \
  --alias gemma3
```

Flag Breakdown:

- `-t 2` : Allocates 2 CPU threads for LLM execution, keeping 2 cores free for IDE/OS responsiveness.
- `-ngl 99` : Offloads all layers to Integrated Intel GPU via Vulkan.
- `-c 4096` : Context window optimal for rapid autocomplete latency.
- `--load-mode mlock` : UPDATED Memory locking. Prevents OS disk swapping (replaces deprecated `--mlock`).
- `--reasoning auto` : NEW Enables automatic reasoning token detection for thinking models (e.g. DeepSeek-R1 / Qwen).

PHASE 4: VS CODE INTEGRATION (CONTINUE EXTENSION)

Configure `~/.continue/config.yaml` in VS Code to point to the local `llama-server` endpoint:

```
models:
  - name: "Gemma 3 4B (ThinkStack)"
    provider: "llama.cpp"
    model: "gemma3"
    apiBase: "http://localhost:9090"
    roles:
      - autocomplete
      - chat
      - edit
```

PHASE 5: QUICK ARGUMENT REFERENCE & FLAG SPECIFICATIONS

Flag / Parameter	Status	Description & Best Practice
`--load-mode mlock`	REPLACES --MLOCK	Pins model weights into host physical RAM, preventing OS swap to SSD/Disk. Crucial for eliminating wake-up delay.
`--reasoning [on|off|auto]`	NEW FLAG	Controls reasoning / thinking output generation. `on` forces reasoning mode, `off` disables it, `auto` detects template support.
`-ngl [num]`	Standard	Number of GPU layers to offload to Vulkan/CUDA/Metal VRAM. Set to `99` or `999` to offload all layers.
`-t [num]`	Standard	CPU threads allocated for matrix multiplication. Best practice: match physical core count (e.g., 2 or 4).
`-c [num]`	Standard	Context window size in tokens (e.g., 4096, 8192, 16384).
`--rpc [nodes]`	Cluster	Comma-separated list of remote RPC worker nodes (e.g., `192.168.1.11:50052,192.168.1.12:50052`).
`-ts, --tensor-split [ratios]`	Cluster	Comma-separated proportional weight split ratios (supports **whole & decimal GB values** like `8,6.5` or `24,24,8`). Passed on Main Machine command.

PHASE 6: THE "THINKSTACK" CLUSTER (DISTRIBUTED COMPUTING & RPC)

Combine multiple network machines (e.g. Main T470 + Worker L470 + Worker T460s) to run large models up to 70B parameters across combined RAM.

1. RPC Server Binary Renaming & Listener Node Setup

> **Binary Renaming Notice:** The RPC listener executable previously named `rpc-server` is now officially renamed to `ggml-rpc-server` in current `llama.cpp` builds.

Launching the Worker Listener Node:
On worker machines, start `ggml-rpc-server` to listen for incoming compute requests across network interface `0.0.0.0` :

```
# Command syntax for worker node (bind to port 50052 across all interfaces):
ggml-rpc-server -H 0.0.0.0 -p 50052 -t 4
```

Alternatively, launch Option 3 "📡 **[Listener Node] Launch ggml-rpc-server**" directly from `custom-LLAMA-CPP-MANAGER` , which provides an interactive TUI setup and elegant SIGINT trap handler:

```
trap 'echo -e "\n\n\e[1;48;5;203m closed \"ggml-rpc-server\" (listener for distributed rem
ggml-rpc-server -H 0.0.0.0 -p 50052 -t 4
```

2. Calculating Tensor Split Ratios on Your Own (3-Step Rule of Thumb)

MISCONCEPTION CLEARING & COMMAND LOCATION:
1. Tensor split numbers (e.g. `3`) are **NOT division denominators**! Passing `3` for a 24GB machine does NOT mean "24GB ÷ 3 = 8GB". Instead, the numbers represent **proportional share weights (parts)** of the total model layers.
2. **Where is `--tensor-split` passed?** `ggml-rpc-server` running on the worker node does NOT take `--tensor-split`. You pass `--tensor-split` on the **Main Machine** when launching `llama-cli` or `llama-server` as a comma-separated list of numbers matching each node target in order: `[Local_Main_GB, Worker1_GB, Worker2_GB]`.

Rule of Thumb: How to calculate ratio numbers for your cluster in 3 steps

Step 1: Write down available RAM/VRAM on each machine
Example: Machine A (Main) has **8GB** desired RAM, Machine B (Worker, 12GB total) has **6.5GB** desired RAM.

Step 2: Simplify or pass GB values directly into `--tensor-split`
Because `llama.cpp` accepts whole or decimal GB values directly, pass `8,6.5`!

Step 3: Run the command on Main Machine
```
llama-cli -m model.gguf --rpc 192.168.1.11:50052 --tensor-split 8,6.5
```

PRO-TIP (Direct GB & Decimal Numbers Shortcut):
Because `llama.cpp` accepts floating-point decimal numbers and normalizes all ratios proportionally, **you can type exact whole or decimal Gigabytes of RAM directly as your tensor split numbers!**
- **Whole GBs:** `--tensor-split 24,24,8` or `--tensor-split 12,24,4`
- **Decimal / Non-Whole GBs:** Want 8GB on Main and 6.5GB on Worker? Simply pass: `--tensor-split 8,6.5`!

Proportion Formula:
$$\text{Node Share } (\%) = \left(\frac{\text{Ratio for Node } i}{\text{Sum of All Node Ratios}} \right) \times 100\%$$

Tensor Split Ratios Reference Table

Cluster Configuration	Input Ratio (`-ts`)	Mathematical Breakdown	Workload Distribution Result
Single Worker Offload (Main Control + 1 Remote Worker)	`0,1`	Main: $0 / 1 = 0\%$ Worker: $1 / 1 = 100\%$	**100% Offload** to Worker node. Main machine uses 0% tensor RAM.
Main + 1 Worker (Decimal GBs) (Main 8GB + Worker 6.5GB out of 12GB)	`8,6.5`	Sum = 14.5 Main: $8 / 14.5 \approx 55.2\%$ Worker: $6.5 / 14.5 \approx 44.8\%$	Allocates **55.2%** of model tensors to Main and **44.8%** to Worker (holding ~6.5GB).
2 Equal Workers (Main 24GB + Worker 24GB)	`1,1` or `24,24`	Main: $1 / 2 = 50\%$ Worker: $1 / 2 = 50\%$	Equal **50% / 50%** split across both machines.
3 Machines (Unequal RAM) (24GB Main + 24GB W1 + 8GB W2)	`3,3,0.8` or `24,24,6.4`	Sum = 6.8 Main: 44.1%, W1: 44.1%, W2: 11.8%	Balanced by RAM capacity. Gives 8GB machine a small load to prevent OOM.

3. Single Worker Node Offloading (100% Remote Offload Example)

If your main machine acts only as a light control node and you wish to offload 100% of tensor operations to a powerful worker node (e.g., `192.168.1.11:50052`):

Use `--tensor-split 0,1` (or `-ts 0,1`):

```
llama-cli \
  -m /path/to/model.gguf \
  --rpc 192.168.1.11:50052 \
  --tensor-split 0,1 \
  -ngl 99 \
  --load-mode mlock \
  --reasoning auto \
  -p "Explain quantum computing in simple terms."
```

4. Multi-Node Cluster Command Example

For a 2-node cluster where Main allocates 8GB and Worker allocates 6.5GB:

```
llama-cli \
  -m /path/to/model.gguf \
  --rpc 192.168.1.11:50052 \
  --tensor-split 8,6.5 \
  -ngl 999 \
  --load-mode mlock \
  -t 4 \
  -p "Running distributed inference with 6.5GB on worker!"
```

PHASE 7: HARDCORE LATENCY & NETWORK TUNING

1. **Boost Network TCP Priority & Sockets (Run on ALL nodes):**

   ```
   sudo sysctl -w net.ipv4.tcp_low_latency=1
   sudo sysctl -w net.core.rmem_max=16777216
   sudo sysctl -w net.core.wmem_max=16777216
   ```

2. **Disable Kernel Memory Swap (Run on ALL nodes):**

   ```
   sudo swapoff -a
   ```

3. **Verify Available Vulkan Memory:**

   ```
   llama-vulkan-check
   ```

PHASE 8: CUSTOM-LLAMA-CPP-MANAGER TUI WORKFLOW

The updated `custom-LLAMA-CPP-MANAGER` TUI automates profile management, listener nodes, downloads, and execution:

- **Option 1: [Load/RUN] Llama Profile** - Launch CLI or Web Server with profile parameters.
- **Option 2: [STOP/List] llama cpp instance(s)** - Detect and kill running `llama-server`, `llama-cli`, or `ggml-rpc-server` processes.
- **Option 3: [Listener Node] Launch ggml-rpc-server** - Turns machine into a worker node for distributed remote RPC requests.

- **Option 4-6: Profile Management** - Create, edit, or delete saved GGUF profile configurations stored in `/bin/CUSTOM_LLAMA_CPP_MANAGER_CONFIG/llama_profiles.conf`.
- **Option 8: Download GGUF Model** - Direct extraction from Hugging Face or Ollama Registry without metadata overhead.